

# CO2 ASSESSMENT TOOL 3 DEBUGGING & OPTIMISATION TASK

**Name:-CH.Harshitha**

**Reg;-192424380**

**Course Code:-CSA0652 Design and Algorithm Analysis For Complexity Analysis**

## **Q18. Debugging Maximum Element (Wrong Comparison Direction)**

### **1. Problem Understanding & Context Analysis**

#### **Problem Statement**

The given program is intended to find the maximum element in an array. However, the comparison operator is reversed, causing the algorithm to return the minimum element instead of the maximum element.

#### **Understanding of the Problem**

#### **Purpose of the Program**

- Find the largest element present in an array.

#### **Expected Input**

arr = [10, 25, 5, 40, 15]

#### **Expected Output**

40

#### **Algorithm Concept Involved**

- Linear Search
- Traverse the array and keep track of the largest element encountered so far.

#### **Location of the Error**

```
if arr[i] < max_val:
```

#### **Root Cause Analysis**

#### **Original Code**

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(len(arr)):  
        if arr[i] < max_val: # ERROR  
            max_val = arr[i]  
    return max_val
```

## Identified Errors

| S.No | Error Line               | Type of Error      | Explanation                                         |
|------|--------------------------|--------------------|-----------------------------------------------------|
| 1    | if arr[i] < max_val      | Logical Error      | Finds the minimum element instead of the maximum    |
| 2    | Loop starts from index 0 | Optimization Issue | First element is compared with itself unnecessarily |

## Why the Error Occurs

The condition updates max\_val whenever a smaller value is found.

## Effect on Output

The algorithm returns the minimum element instead of the maximum element.

## 2. Application of Concepts

### Original Code

```
def find_max(arr):
    max_val = arr[0]
    for i in range(len(arr)):
        if arr[i] < max_val:
            max_val = arr[i]
    return max_val
```

## Identified Errors

| S.No | Error Line          | Type of Error      | Explanation                         |
|------|---------------------|--------------------|-------------------------------------|
| 1    | if arr[i] < max_val | Logical Error      | Wrong comparison operator           |
| 2    | range(len(arr))     | Optimization Issue | Extra comparison with first element |

## Corrected Logic

### Correction Applied

```
if arr[i] > max_val:
```

### Algorithm Rules Followed

- Update maximum only when a larger element is found.
- Check every element exactly once.

## How Correctness is Restored

The variable max\_val always stores the largest element encountered so far.

### **3. Analytical & Decision-Making Skills**

#### **Debugging Process**

##### **Step 1**

Inspect the comparison condition used for updating the maximum value.

##### **Step 2**

Test the code with sample arrays and observe that it returns the smallest value.

##### **Step 3**

Replace < with > and verify the output.

#### **Optimization Applied**

- Start traversal from index 1.
- Remove unnecessary comparison with the first element.

#### **Justification**

##### **Why the Corrected Algorithm is Accurate**

Every element is compared against the current maximum, ensuring the largest element is found.

##### **Why the Optimization Improves Performance**

One unnecessary comparison is eliminated while maintaining the same complexity.

### **4. Solution Design & Approach**

#### **Corrected & Optimized Code**

```
def find_max(arr):
    if len(arr) == 0:
        return None

    max_val = arr[0]

    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]

    return max_val

arr = [10, 25, 5, 40, 15]
print("Maximum Element:", find_max(arr))
```

### Algorithm / Logic Flow

1. Start the program.
2. Read the array.
3. Initialize the first element as maximum.
4. Traverse the remaining elements.
5. Compare each element with the current maximum.
6. Update maximum if a larger element is found.
7. Return the maximum element.
8. Stop.

### 5. Clarity, Justification & Presentation

#### Time Complexity Analysis

| Case         | Complexity |
|--------------|------------|
| Best Case    | $O(n)$     |
| Average Case | $O(n)$     |
| Worst Case   | $O(n)$     |

#### Space Complexity

$O(1)$

#### Sample Input

arr = [10, 25, 5, 40, 15]

#### Expected Output

Maximum Element: 40

#### Final Conclusion

- The comparison operator was reversed.
- The algorithm incorrectly returned the minimum element.
- Replacing  $<$  with  $>$  corrected the logic.
- Starting the loop from index 1 removed an unnecessary comparison.
- The corrected algorithm successfully finds the maximum element.

### Execution:-

main.py

 Share

Run

```
1 def find_max(arr):
2     max_val = arr[0]
3     for i in range(1, len(arr)):
4         if arr[i] > max_val:
5             max_val = arr[i]
6     return max_val
7 arr = [10, 25, 5, 40, 15]
8 print("Maximum element:", find_max(arr))
```

### Output:-

Output

Maximum element: 40

=== Code Execution Successful ===

## **Q34. Debugging Binary Search Code (Incorrect Mid Recalculation Logic)**

### **1. Problem Understanding & Context Analysis**

#### **Problem Statement**

The given Binary Search algorithm updates the search boundaries incorrectly, causing valid portions of the array to be discarded.

#### **Understanding of the Problem**

#### **Purpose of the Program**

- Search for a target element in a sorted array using Binary Search.

#### **Expected Input**

arr = [5, 10, 15, 20, 25, 30]

key = 20

#### **Expected Output**

3

#### **Algorithm Concept Involved**

- Binary Search
- Divide the search space into two halves after each comparison.

#### **Location of the Error**

```
elif arr[mid] > key:
```

```
    low = mid + 1
```

```
else:
```

```
    high = mid - 1
```

#### **Root Cause Analysis**

#### **Original Code**

```
def binary_search(arr, key):
```

```
    low, high = 0, len(arr)-1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == key:
```

```
            return mid
```

```
        elif arr[mid] > key:
```

```
low = mid + 1
```

```
else:
```

```
high = mid - 1
```

```
return -1
```

### Identified Errors

| S.No | Error Line     | Type of Error | Explanation                         |
|------|----------------|---------------|-------------------------------------|
| 1    | low = mid + 1  | Logical Error | Searches right side instead of left |
| 2    | high = mid - 1 | Logical Error | Searches left side instead of right |

### Why the Error Occurs

The search boundaries are updated in the wrong direction.

### Effect on Output

Valid portions of the array are discarded and the target may never be found.

## 2. Application of Concepts

### Original Code

```
def binary_search(arr, key):  
    low, high = 0, len(arr)-1
```

```
    while low <= high:  
        mid = (low + high) // 2
```

```
        if arr[mid] == key:  
            return mid
```

```
        elif arr[mid] > key:  
            low = mid + 1
```

```
        else:  
            high = mid - 1
```

```
    return -1
```

## Identified Errors

| S.No | Error Line     | Type of Error | Explanation            |
|------|----------------|---------------|------------------------|
| 1    | low = mid + 1  | Logical Error | Wrong direction update |
| 2    | high = mid - 1 | Logical Error | Wrong direction update |

## Corrected Logic

### Correction Applied

```
elif arr[mid] > key:  
    high = mid - 1
```

```
else:  
    low = mid + 1
```

### Algorithm Rules Followed

- If middle element is greater than key, search left half.
- If middle element is smaller than key, search right half.

### How Correctness is Restored

The search space shrinks correctly while retaining all possible positions of the key.

## 3. Analytical & Decision-Making Skills

### Debugging Process

#### Step 1

Trace the execution using a sample sorted array.

#### Step 2

Observe that boundary updates move in the wrong direction.

#### Step 3

Correct the updates and verify the output.

### Optimization Applied

- Clear midpoint calculation.
- Proper boundary updates.
- Constant extra memory usage.



## **Justification**

### **Why the Corrected Algorithm is Accurate**

Each iteration removes half of the remaining search space while preserving the correct search region.

### **Why the Optimization Improves Performance**

Binary Search reduces the number of comparisons significantly compared to Linear Search.

## **4. Solution Design & Approach**

### **Corrected & Optimized Code**

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] > key:
            high = mid - 1

        else:
            low = mid + 1

    return -1

arr = [5, 10, 15, 20, 25, 30]
key = 20

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

### **Algorithm / Logic Flow**

1. Start the program.

2. Read the sorted array and key.
3. Initialize low and high.
4. Calculate middle index.
5. Compare middle element with key.
6. If equal, return index.
7. If greater, search left half.
8. If smaller, search right half.
9. Repeat until found or search space becomes empty.
10. Return -1 if not found.
11. Stop.

## 5. Clarity, Justification & Presentation

### Time Complexity Analysis

| Case         | Complexity  |
|--------------|-------------|
| Best Case    | $O(1)$      |
| Average Case | $O(\log n)$ |
| Worst Case   | $O(\log n)$ |

### Space Complexity

$O(1)$

### Sample Input

arr = [5, 10, 15, 20, 25, 30]

key = 20

### Expected Output

Element found at index 3

### Final Conclusion

- The boundary updates were reversed.
- The algorithm searched the wrong half of the array.
- Correcting the updates restored proper Binary Search functionality.
- The final solution works correctly with  $O(\log n)$  time complexity and  $O(1)$  space complexity.

## Execution:-

```
main.py  [Full Screen] [Dark Mode] [Share] [Run]

1 def binary_search(arr, key):
2     low, high = 0, len(arr) - 1
3     while low <= high:
4         mid = (low + high) // 2
5         if arr[mid] == key:
6             return mid
7         elif arr[mid] > key:
8             high = mid - 1
9         else:
10            low = mid + 1
11    return -1
12 # Example
13 arr = [10, 20, 30, 40, 50, 60, 70]
14 key = 20
15 result = binary_search(arr, key)
16 if result != -1:
17     print("Element found at index:", result)
18 else:
19     print("Element not found")
```

## Output:-

```
Output

Element found at index: 1

=== Code Execution Successful ===
```